Task – II: Product and Category Management System

1. Objective

Design and implement a database module to manage products and their categories in an E-Commerce Order Management System.

2. Database Tables

Category Table

Field	Data Type	Constraint	Description
category_id	INT	Primary Key	Unique category ID
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the category
description	VARCHAR(255)	—	Category description

Product Table

Field	Data Type	Constraint	Description
product_id	INT	Primary Key	Unique product ID
product_name	VARCHAR(150)	NOT NULL	Name of the product
category_id	INT	Foreign Key	References Category
price	DECIMAL(10,2)	NOT NULL	Product price
stock	INT	NOT NULL	Available quantity

3. Primary and Foreign Key Relationship

• category_id is the Primary Key in the Category table.

• product_id is the Primary Key in the Product table.

• Product.category_id is a Foreign Key referencing Category.category_id.

• One category can contain many products (1:N relationship).

4. SQL Table Creation

```
CREATE TABLE Category (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255)
);

CREATE TABLE Product (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(150) NOT NULL,
    category_id INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL,
    FOREIGN KEY (category_id)
        REFERENCES Category(category_id)
```

```
);
```

5. Product Insertion

```
INSERT INTO Category (category_name, description)
VALUES ('Electronics', 'Electronic products');

INSERT INTO Product (product_name, category_id, price, stock)
VALUES ('Wireless Headphones', 1, 2499.00, 50);
```

6. Product Updating

```
UPDATE Product
SET price = 2299.00,
    stock = 45
WHERE product_id = 1;
```

7. Product Deletion

```
DELETE FROM Product
WHERE product_id = 1;
```

8. Category-Wise Product Report

```
SELECT
    c.category_name,
    p.product_id,
    p.product_name,
    p.price,
    p.stock
FROM Category c
JOIN Product p
ON c.category_id = p.category_id
ORDER BY c.category_name;
```

9. Category Summary Report

```
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_products,
    SUM(p.stock) AS total_stock,
    AVG(p.price) AS average_price
FROM Category c
LEFT JOIN Product p
ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name;
```

10. Expected Outcome

• Category creation and management

• Product insertion, updating, and deletion

• Stock management

• Product–category relationships

• Category-wise product reports

• Product price and inventory tracking